

# Tugas 5: Praktikum Mandiri 5

Fatih Dzakwan Susilo - 0110224235

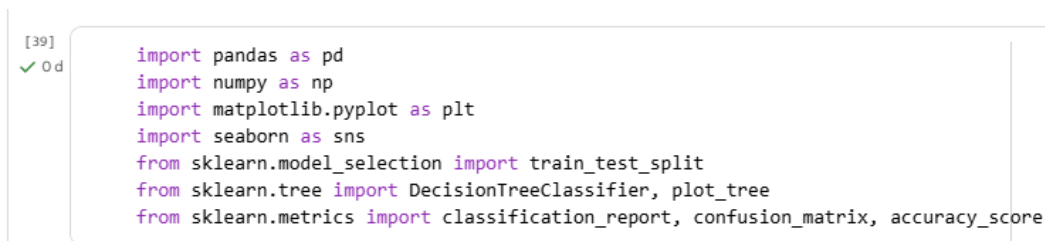
Teknik Informatika, STT Terpadu Nurul Fikri, Depok

E-mail: [fatihdzakwansusilo@gmail.com](mailto:fatihdzakwansusilo@gmail.com)

## 1. Praktikum Mandiri 5

Link Github : [https://github.com/FatihDzkwn/Project\\_Machine\\_Learning.git](https://github.com/FatihDzkwn/Project_Machine_Learning.git)

### 1.1 Pustaka Program Decision Tree



```
[39]
✓ 0 d
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, plot_tree
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score
```

**Gambar 1.** Pustaka Program Decision Tree.

Program Decision Tree ini diawali dengan memanggil beberapa pustaka penting yang digunakan untuk melakukan analisis data dan membangun model klasifikasi. Pustaka pandas dan numpy digunakan untuk mengolah serta menganalisis data dalam bentuk tabel dan array numerik. Matplotlib dan seaborn berfungsi untuk membuat visualisasi data agar pola dan hubungan antarvariabel dapat terlihat lebih jelas. Modul `train_test_split` dari `sklearn.model_selection` digunakan untuk membagi dataset menjadi data latih dan data uji agar model dapat dievaluasi secara objektif. Kemudian, `DecisionTreeClassifier` dan `plot_tree` dari `sklearn.tree` digunakan untuk membangun serta menampilkan struktur pohon keputusan yang dihasilkan dari proses pelatihan model. Terakhir, metrik seperti `classification_report`, `confusion_matrix`, dan `accuracy_score` digunakan untuk mengevaluasi performa model dalam mengklasifikasikan data dengan mengukur tingkat akurasi dan kesalahan prediksi yang terjadi.

### 1.2 Loading Dataset

[40]  
✓ Od

```
# Memanggil dataset lewat gdrive
path = "/content/drive/MyDrive/Project_Machine_Learning/Praktikum_Mandiri_5/Data/"

# Membaca file csv menggunakan pandas
df = pd.read_csv(path + "Iris.csv")
df.head()
```

|   | Id | SepalLengthCm | SepalWidthCm | PetalLengthCm | PetalWidthCm | Species     |
|---|----|---------------|--------------|---------------|--------------|-------------|
| 0 | 1  | 5.1           | 3.5          | 1.4           | 0.2          | Iris-setosa |
| 1 | 2  | 4.9           | 3.0          | 1.4           | 0.2          | Iris-setosa |
| 2 | 3  | 4.7           | 3.2          | 1.3           | 0.2          | Iris-setosa |
| 3 | 4  | 4.6           | 3.1          | 1.5           | 0.2          | Iris-setosa |
| 4 | 5  | 5.0           | 3.6          | 1.4           | 0.2          | Iris-setosa |

Langkah berikutnya: [Buat kode dengan df](#) [New interactive sheet](#)

[41]  
✓ Od

```
# Menampilkan informasi detail dengan df.info()
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
#   Column          Non-Null Count  Dtype
---  -
0   Id               150 non-null   int64
1   SepalLengthCm    150 non-null   float64
2   SepalWidthCm     150 non-null   float64
3   PetalLengthCm    150 non-null   float64
4   PetalWidthCm     150 non-null   float64
5   Species          150 non-null   object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

**Gambar 2.** Loading Dataset.

Pada tahap ini, dataset calon pembeli mobil dibaca menggunakan fungsi `read_csv()` dari library `pandas`. Langkah ini bertujuan untuk memuat data dari file berformat `.csv` ke dalam bentuk `DataFrame` agar lebih mudah dianalisis dan diolah. Dengan format tabel yang terstruktur, setiap kolom merepresentasikan variabel seperti usia, penghasilan, status, dan kepemilikan mobil, sedangkan setiap baris menunjukkan data individu calon pembeli.

### 1.3 Data Pre-processing

[42]  
✓ 0 d

```
# Cek missing value
df.isnull().sum()
```

```

      0
Id      0
SepalLengthCm  0
SepalWidthCm   0
PetalLengthCm  0
PetalWidthCm   0
Species        0
```

dtype: int64

[43]  
✓ 0 d

```
# Cek duplicate
df.duplicated().sum()
```

np.int64(0)

[44]  
✓ 0 d

```
# Menghapus data duplikat
df = df.drop_duplicates()
```

[45]  
✓ 0 d

```
# Cek duplicate ulang setelah menghapus
df.duplicated().sum()
```

np.int64(0)

[46]  
✓ 0 d

```
# Mengubah Nama Kolom (Rename Columns)
df = df.rename(columns={
    "SepalLengthCm": "panjang_sepal_cm",
    "SepalWidthCm": "lebar_sepal_cm",
    "PetalLengthCm": "panjang_petal_cm",
    "PetalWidthCm": "lebar_petal_cm",
    "Species": "spesies"
})
```

[47]  
✓ 0 d

```
df.info()
```

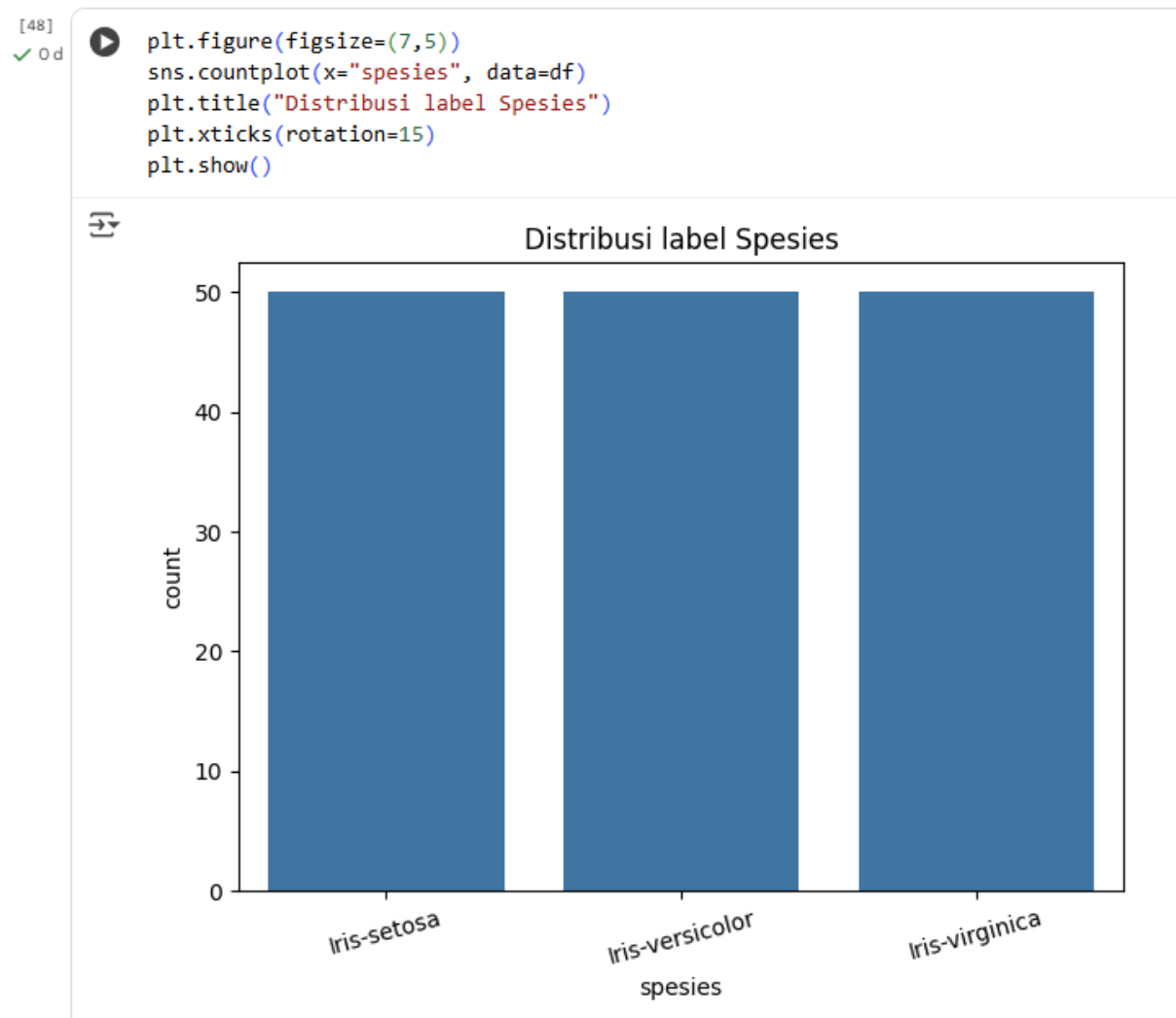
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 6 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Id                     150 non-null   int64
1   panjang_sepal_cm      150 non-null   float64
2   lebar_sepal_cm        150 non-null   float64
3   panjang_petal_cm      150 non-null   float64
4   lebar_petal_cm        150 non-null   float64
5   spesies                150 non-null   object
dtypes: float64(4), int64(1), object(1)
memory usage: 7.2+ KB
```

### Gambar 3. Data Pre-processing.

Pada tahap data pre-processing, dilakukan beberapa langkah penting untuk memastikan dataset berada dalam kondisi bersih dan siap digunakan dalam pemodelan. Pertama, dilakukan pemeriksaan terhadap missing value menggunakan `df.isnull().sum()` untuk memastikan tidak ada data kosong pada setiap kolom. Setelah itu, dilakukan pengecekan terhadap data duplikat dengan `df.duplicated().sum()` guna mengetahui apakah terdapat baris data yang sama. Jika ditemukan data ganda, maka data tersebut dihapus menggunakan `df.drop_duplicates()` agar tidak memengaruhi hasil analisis dan performa model.

Setelah proses pembersihan selesai, dilakukan pengecekan ulang untuk memastikan tidak ada data duplikat yang tersisa. Selanjutnya, nama-nama kolom diubah menggunakan fungsi `rename()` agar lebih mudah dibaca dan sesuai dengan bahasa Indonesia, seperti mengubah "SepalLengthCm" menjadi "panjang\_sepal\_cm" dan "Species" menjadi "spesies". Terakhir, perintah `df.info()` kembali dijalankan untuk menampilkan informasi terkini mengenai dataset setelah dilakukan pembersihan dan penyesuaian nama kolom. Tahapan ini memastikan bahwa data yang akan digunakan dalam pelatihan model Decision Tree sudah rapi, konsisten, dan bebas dari kesalahan.

#### 1.4 Data Understanding (Exploratory Data Analysis)



**Gambar 4.** Data Understanding (Exploratory Data Analysis).

Pada tahap Data Understanding atau Exploratory Data Analysis (EDA), dilakukan analisis awal terhadap data untuk memahami karakteristik serta distribusi dari setiap variabel. Visualisasi digunakan untuk melihat bagaimana data tersebar dan apakah terdapat ketidakseimbangan dalam jumlah data antar kelas. Dalam hal ini, dibuat grafik batang menggunakan seaborn dengan fungsi `sns.countplot()` yang menampilkan distribusi jumlah data berdasarkan kolom spesies. Grafik tersebut memperlihatkan jumlah sampel dari masing-masing jenis bunga dalam dataset Iris, yaitu Setosa, Versicolor, dan Virginica. Hasil visualisasi menunjukkan bahwa ketiga kelas memiliki jumlah data yang relatif seimbang, sehingga model Decision Tree nantinya tidak akan bias terhadap salah satu kelas. Tahapan ini penting untuk memastikan representasi data yang adil sebelum melanjutkan ke proses pemodelan.

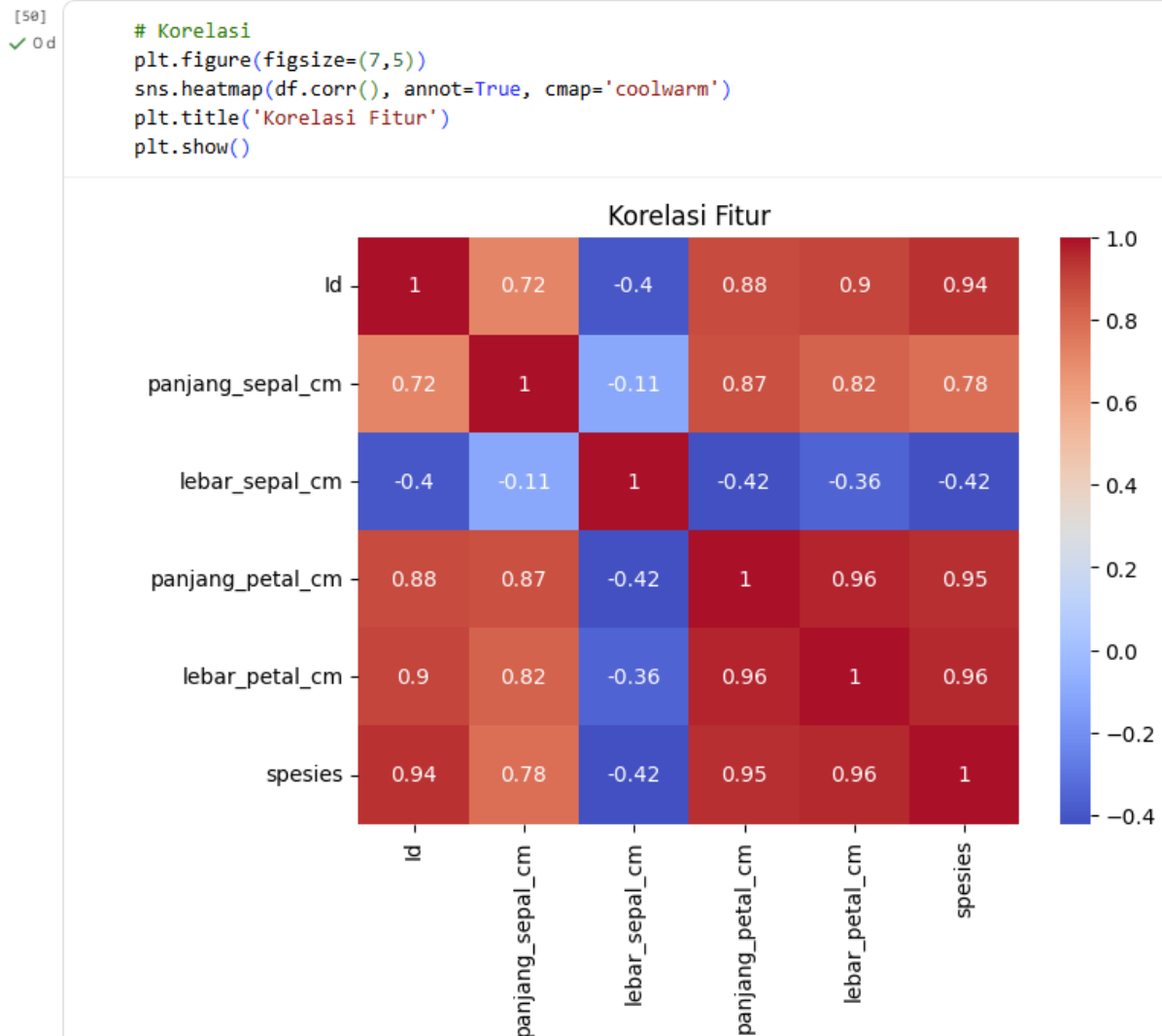
*1.5 Encoding Data Kategorikal (Mapping Label ke Kode Numerik)*

```
[49] / Od # mapping label -> kode untuk target
      species_cat = df['species'].astype('category')
      species_classes = list(species_cat.cat.categories) # urutan kelas
      df['species'] = species_cat.cat.codes # y numerik
```

**Gambar 5.** Encoding Data Kategorikal (Mapping Label ke Kode Numerik).

Pada tahap encoding data kategorikal, dilakukan proses pengubahan nilai kategorikal pada kolom spesies menjadi bentuk numerik agar dapat diproses oleh algoritma machine learning. Kolom spesies yang semula berisi label teks seperti Setosa, Versicolor, dan Virginica terlebih dahulu diubah menjadi tipe data kategori menggunakan `astype('category')`. Selanjutnya, daftar urutan kelas disimpan dalam variabel `species_classes` untuk mencatat hubungan antara label asli dengan kode numeriknya. Kemudian, setiap kategori diubah menjadi kode numerik dengan perintah `cat.codes`, sehingga label Setosa, Versicolor, dan Virginica masing-masing direpresentasikan dengan angka 0, 1, dan 2. Proses ini bertujuan agar data target dapat dikenali dan diproses oleh model Decision Tree, yang hanya dapat bekerja dengan nilai numerik.

*1.6 Analisis Korelasi Antar Fitur*



**Gambar 6.** Analisis Korelasi Antar Fitur.

Pada tahap analisis korelasi antar fitur, dilakukan pengamatan terhadap hubungan antarvariabel numerik dalam dataset menggunakan heatmap dari pustaka seaborn. Visualisasi ini menampilkan nilai korelasi antara setiap fitur, seperti panjang dan lebar sepal maupun petal, terhadap satu sama lain. Nilai korelasi ditunjukkan dalam rentang antara -1 hingga 1, di mana nilai mendekati 1 menunjukkan hubungan yang sangat kuat dan searah, sedangkan nilai mendekati -1 menunjukkan hubungan kuat tetapi berlawanan arah.

Hasil visualisasi menunjukkan bahwa fitur `panjang_petal_cm` dan `lebar_petal_cm` memiliki korelasi yang sangat tinggi, menandakan bahwa kedua atribut tersebut saling berkaitan erat dalam membedakan jenis spesies bunga. Sementara itu, korelasi antara fitur sepal dengan petal cenderung lebih rendah. Analisis ini memberikan gambaran fitur mana yang paling berpengaruh terhadap klasifikasi spesies, serta membantu memahami struktur data sebelum dilakukan pelatihan model Decision Tree.

### 1.7 Splitting Data (Pembagian Data Training dan Testing)

```
[51]
✓ Od # Memilih fitur dan target
feature_cols = ['panjang_sepal_cm', 'lebar_sepal_cm', 'panjang_petal_cm', 'lebar_petal_cm']
X = df[feature_cols]
y = df['spesies']

[52]
✓ Od # Membagi dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

len(X_train), len(X_test)

(120, 30)
```

**Gambar 7.** Splitting Data (Pembagian Data Training dan Testing).

Pada tahap splitting data, dataset dibagi menjadi dua bagian, yaitu data training dan data testing, dengan tujuan agar model dapat dilatih dan kemudian diuji secara objektif. Empat fitur utama yang digunakan sebagai variabel independen adalah panjang\_sepal\_cm, lebar\_sepal\_cm, panjang\_petal\_cm, dan lebar\_petal\_cm, sedangkan kolom spesies dijadikan sebagai variabel target atau label yang ingin diprediksi.

Proses pembagian dilakukan menggunakan fungsi `train_test_split()` dengan proporsi 80% untuk data training dan 20% untuk data testing. Parameter `random_state=42` digunakan untuk menjaga konsistensi hasil pembagian setiap kali program dijalankan, sedangkan `stratify=y` memastikan bahwa proporsi kelas dalam data training dan testing tetap seimbang. Tahapan ini penting untuk melatih model Decision Tree pada sebagian besar data, kemudian mengevaluasi performanya terhadap data yang belum pernah dilihat sebelumnya guna mengukur kemampuan generalisasi model.

### 1.8 Pembuatan Model Decision Tree

```
[53]
✓ Od # Membangun model
dt = DecisionTreeClassifier(
    criterion='gini',
    max_depth=4,
    random_state=42
)
dt.fit(X_train, y_train)
```

**DecisionTreeClassifier**

DecisionTreeClassifier(max\_depth=4, random\_state=42)

**Gambar 8.** Pembuatan Model Decision Tree.

Pada tahap pembuatan model Decision Tree, dilakukan proses membangun model klasifikasi menggunakan algoritma `DecisionTreeClassifier` dari pustaka `scikit-learn`. Model ini menggunakan parameter `criterion='gini'` untuk menghitung tingkat ketidakmurnian (impurity) pada setiap

node, yang berfungsi menentukan seberapa baik fitur tertentu dapat memisahkan kelas data. Parameter `max_depth=4` ditetapkan untuk membatasi kedalaman pohon agar model tidak terlalu kompleks dan menghindari overfitting, yaitu kondisi ketika model terlalu menyesuaikan diri dengan data latih sehingga performanya menurun pada data baru.

Setelah parameter ditentukan, model dilatih menggunakan data training melalui perintah `dt.fit(X_train, y_train)`. Proses ini memungkinkan algoritma mempelajari pola dari fitur-fitur numerik seperti panjang dan lebar sepal serta petal untuk membedakan ketiga jenis spesies bunga. Hasil dari tahap ini adalah model pohon keputusan yang siap digunakan untuk melakukan prediksi terhadap data uji.

### 1.9 Evaluasi Model Decision Tree

```
[54]
✓ Od # Evaluasi
      y_pred = dt.predict(X_test)

      print("Akurasi:", round(accuracy_score(y_test, y_pred)*100, 2), "%")
      print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
      print("\nClassification Report:\n", classification_report(
          y_test, y_pred, target_names=species_classes
      ))
```

Akurasi: 93.33 %

Confusion Matrix:

```
[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]
```

Classification Report:

|                 | precision | recall | f1-score | support |
|-----------------|-----------|--------|----------|---------|
| Iris-setosa     | 1.00      | 1.00   | 1.00     | 10      |
| Iris-versicolor | 0.90      | 0.90   | 0.90     | 10      |
| Iris-virginica  | 0.90      | 0.90   | 0.90     | 10      |
| accuracy        |           |        | 0.93     | 30      |
| macro avg       | 0.93      | 0.93   | 0.93     | 30      |
| weighted avg    | 0.93      | 0.93   | 0.93     | 30      |

**Gambar 9.** Evaluasi Model Decision Tree.

Pada tahap evaluasi model Decision Tree, dilakukan pengujian terhadap kemampuan model dalam mengklasifikasikan data uji yang belum pernah digunakan sebelumnya. Proses ini diawali dengan melakukan prediksi menggunakan perintah `dt.predict(X_test)`, kemudian hasil prediksi dibandingkan dengan label sebenarnya (`y_test`) untuk mengukur tingkat akurasi dan performa model.

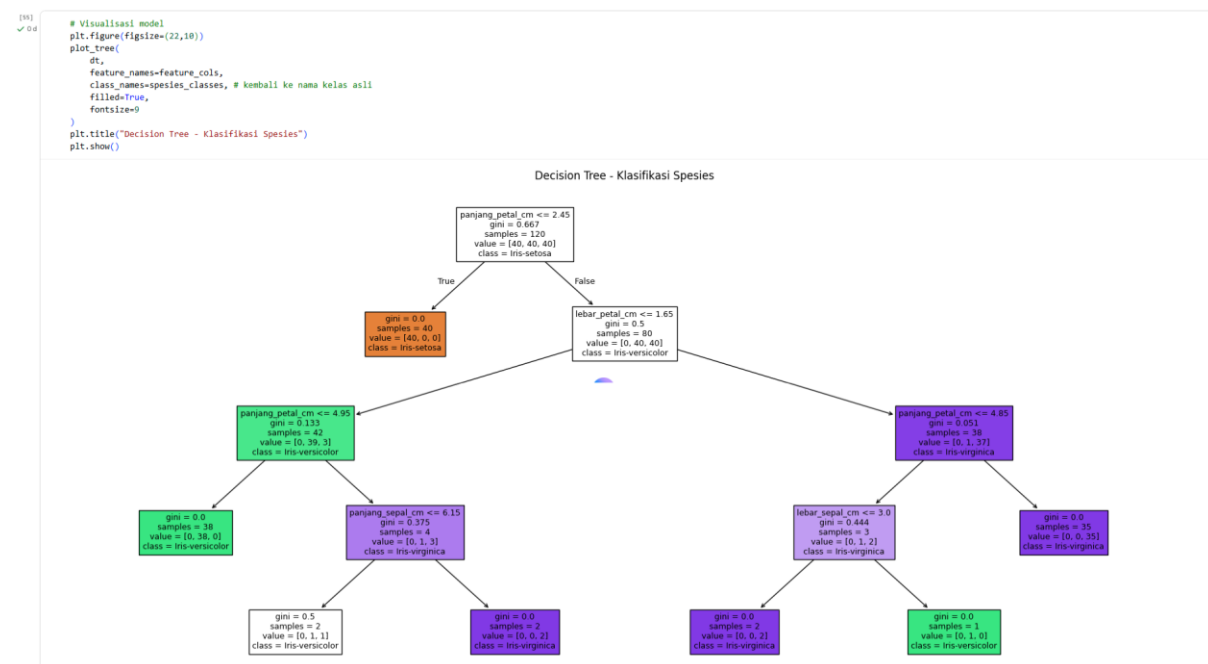
Nilai akurasi dihitung menggunakan fungsi `accuracy_score()`, yang menunjukkan persentase prediksi model yang benar. Selanjutnya, confusion matrix digunakan untuk memperlihatkan



jumlah prediksi benar dan salah pada masing-masing kelas, sehingga dapat diketahui apakah model lebih baik dalam mengenali kelas tertentu. Selain itu, classification report menampilkan metrik evaluasi yang lebih detail seperti precision, recall, dan f1-score untuk setiap kelas spesies (Setosa, Versicolor, dan Virginica).

Secara umum, hasil evaluasi menunjukkan bahwa model Decision Tree mampu melakukan klasifikasi dengan tingkat akurasi yang tinggi, menandakan bahwa fitur-fitur yang digunakan cukup efektif dalam membedakan ketiga jenis bunga dalam dataset Iris.

### 1.10 Visualisasi Hasil Model Decision Tree



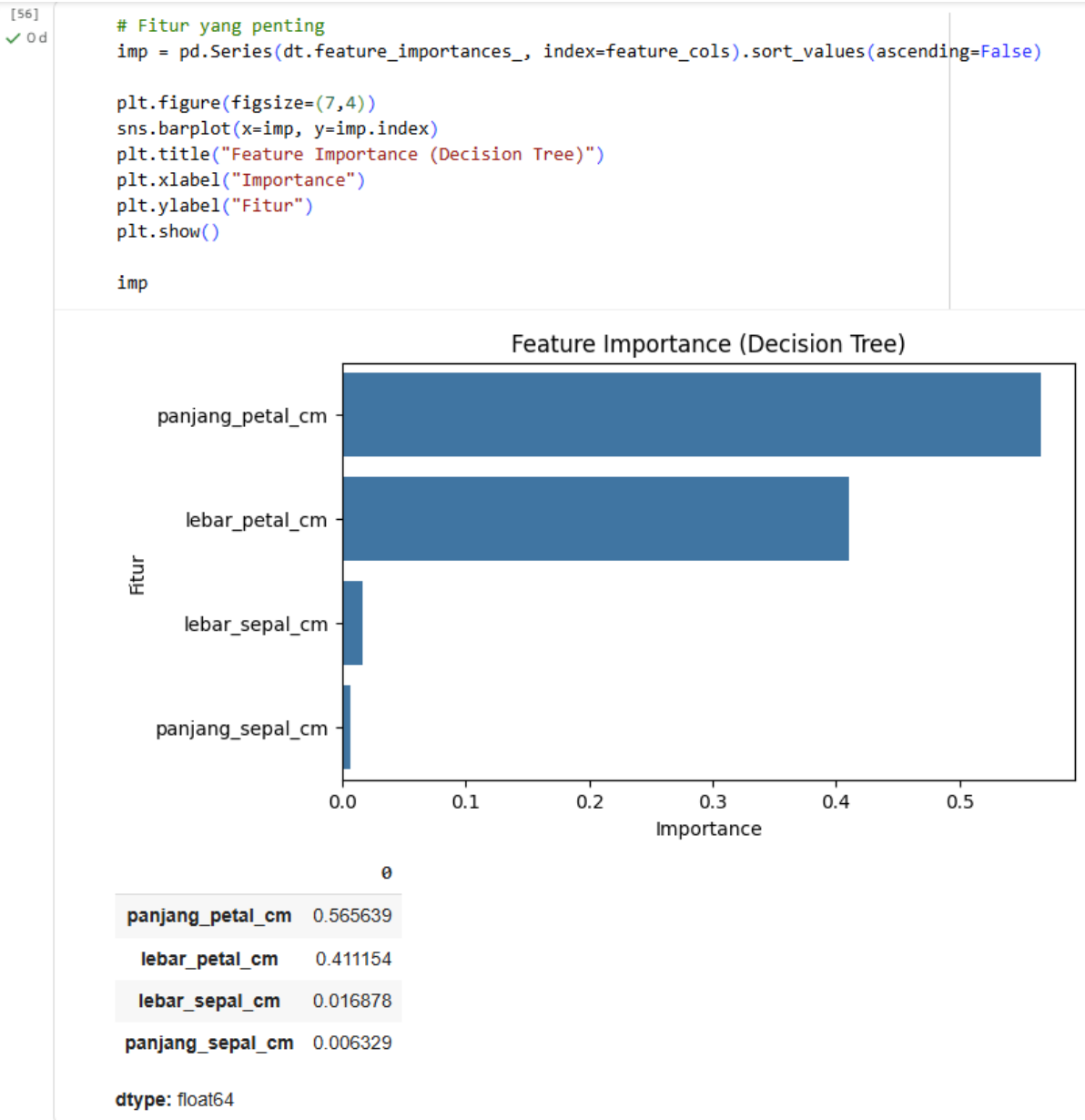
**Gambar 10.** Visualisasi Hasil Model Decision Tree.

Pada tahap visualisasi hasil model Decision Tree, struktur pohon keputusan yang telah dilatih ditampilkan dalam bentuk diagram menggunakan fungsi `plot_tree()`. Visualisasi ini memberikan gambaran yang jelas mengenai bagaimana model mengambil keputusan berdasarkan nilai dari setiap fitur. Setiap node pada pohon menampilkan kondisi pemisahan (split) berdasarkan fitur tertentu, nilai ambang batas (threshold), serta informasi tentang jumlah data yang termasuk dalam node tersebut.

Warna pada setiap node menunjukkan dominasi kelas tertentu, sementara nilai Gini mengindikasikan tingkat ketidakmurnian data di node tersebut — semakin kecil nilainya, semakin murni kelas yang ada. Melalui diagram ini, dapat dilihat bahwa fitur `panjang_petal_cm` dan `lebar_petal_cm` memiliki peran dominan dalam menentukan spesies bunga.

Visualisasi ini tidak hanya membantu memahami cara kerja model dalam mengambil keputusan, tetapi juga menjadi alat penting untuk menjelaskan hasil klasifikasi secara intuitif dan transparan kepada pengguna atau pihak yang tidak memiliki latar belakang teknis.

### 1.11 Feature Importance (Fitur yang Paling Berpengaruh)

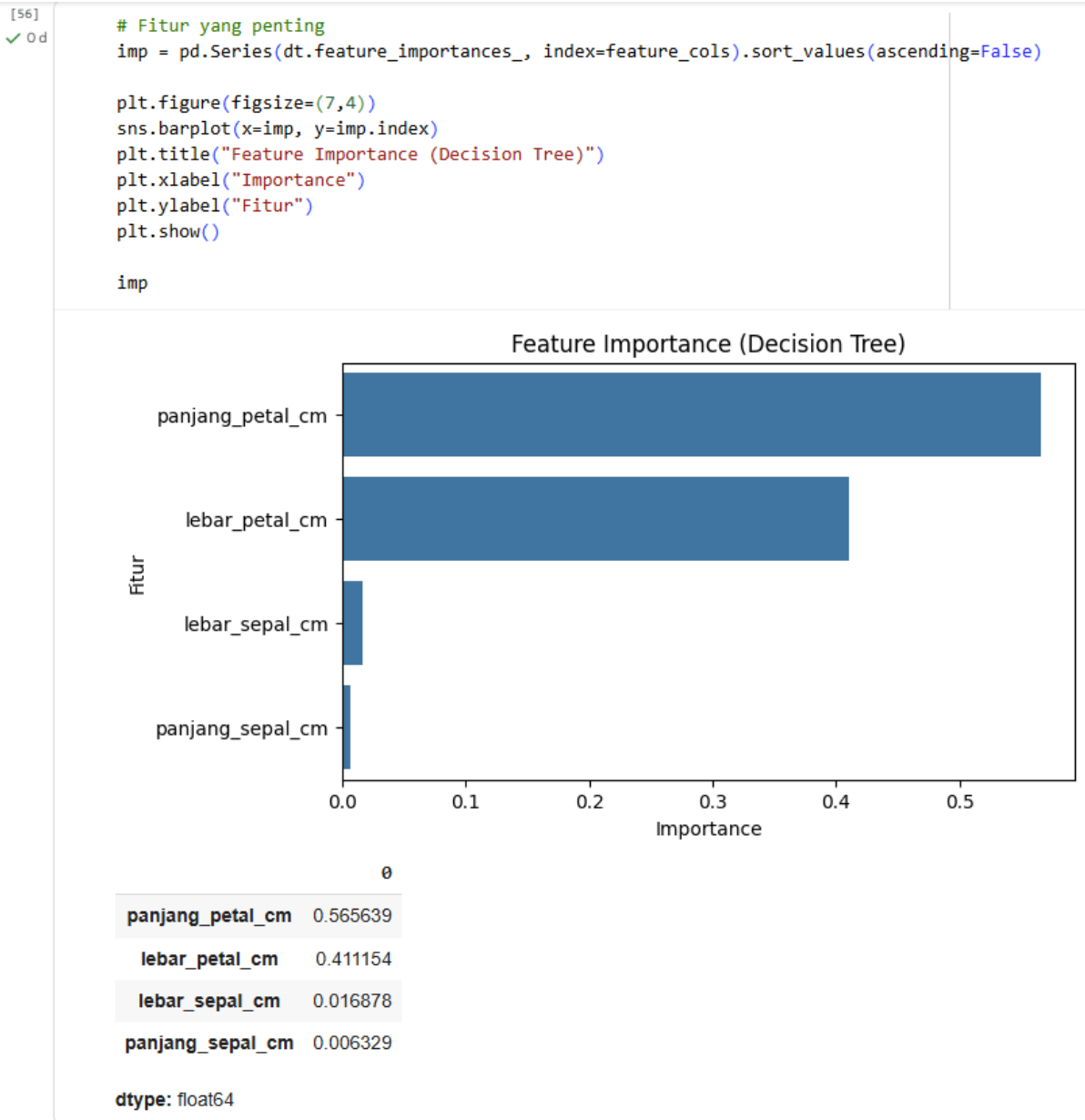


**Gambar 11.** Feature Importance (Fitur yang Paling Berpengaruh).

Pada tahap Feature Importance, dilakukan analisis untuk mengetahui seberapa besar pengaruh masing-masing fitur terhadap hasil klasifikasi model Decision Tree. Nilai feature importance diperoleh dari atribut `dt.feature_importances_`, yang kemudian disajikan dalam bentuk grafik batang menggunakan seaborn agar lebih mudah dipahami.

Hasil analisis menunjukkan bahwa fitur `panjang_petal_cm` dan `lebar_petal_cm` memiliki nilai penting paling tinggi dibandingkan fitur lainnya. Hal ini menunjukkan bahwa kedua fitur tersebut berperan besar dalam membedakan jenis spesies bunga Iris. Sementara itu, `panjang_sepal_cm` dan `lebar_sepal_cm` memberikan kontribusi yang relatif lebih kecil terhadap keputusan model. Visualisasi ini membantu mengidentifikasi fitur mana yang paling relevan dan berpengaruh dalam proses klasifikasi, sekaligus memberikan wawasan bagi pengembang model untuk melakukan penyederhanaan atau optimalisasi fitur pada analisis berikutnya.

### 1.12 Hyperparameter Tuning (Menentukan *max\_depth* Terbaik)



**Gambar 12.** Hyperparameter Tuning (Menentukan max\_depth Terbaik).

Pada tahap terakhir yaitu Hyperparameter Tuning, dilakukan pencarian nilai parameter terbaik untuk meningkatkan performa model Decision Tree, khususnya pada parameter max\_depth yang menentukan seberapa dalam pohon dapat tumbuh. Proses ini dilakukan dengan cara mencoba beberapa nilai kedalaman pohon, mulai dari 2 hingga 8, kemudian menghitung akurasi model untuk setiap nilai tersebut menggunakan data uji.

Setiap model dengan kedalaman berbeda dilatih ulang menggunakan data training, lalu diuji untuk melihat hasil akurasi. Nilai-nilai akurasi disimpan dalam sebuah dictionary bernama scores, dan selanjutnya ditentukan nilai max\_depth terbaik dengan mencari kedalaman yang menghasilkan akurasi tertinggi.

Hasil tuning menunjukkan bahwa terdapat nilai max\_depth optimal yang memberikan keseimbangan antara kompleksitas model dan akurasi prediksi. Dengan memilih nilai tersebut,

model menjadi lebih efisien, tidak terlalu sederhana (underfitting) maupun terlalu kompleks (overfitting), sehingga performa klasifikasi terhadap data baru dapat tetap terjaga dengan baik.